

Plateforme de streaming musical — Défense du design

Contexte

Ce document présente et défend les choix de conception réalisés pour une plateforme de streaming musical simplifiée. Le système permet aux utilisateurs de parcourir de la musique, de gérer des playlists personnelles et de suivre des artistes. Trois cas d'utilisation ont été modélisés sous forme de diagrammes de séquence : **créer une playlist**, **ajouter une chanson à une playlist** et **suivre un artiste**.

1. Principaux choix de conception

Une classe façade unique : `StreamingPlatform`

Plutôt que de modéliser une architecture frontend/backend séparée, nous avons choisi une classe `StreamingPlatform` unique jouant le rôle de **façade** (patron de conception Facade). Elle reçoit toutes les requêtes du formulaire web via des appels API, effectue la validation des entrées et délègue aux classes de domaine concernées.

Cette simplification est un choix délibéré : l'objectif était de garder le modèle lisible et centré sur la logique métier, sans s'attarder sur les couches d'infrastructure. Dans une application réelle, cette classe serait découpée en contrôleurs, services et repositories.

Méthodes statiques `create` et `get` sur chaque classe de données

Chaque classe de domaine (`Song`, `Album`, `Artist`, `User`, `Playlist`) expose :

- une méthode statique `+create(...) $ int` — encapsule la logique d'instanciation et retourne l'identifiant technique du nouvel enregistrement
- une méthode statique `+get(id: int) $ T` — encapsule la récupération par clé primaire

Il s'agit d'un **patron Repository simplifié** : chaque classe est responsable de ses propres opérations de persistance. Dans une architecture en couches, ces méthodes vivraient dans des classes dédiées (`SongRepository`), (`PlaylistRepository`), etc., mais ce niveau de séparation était hors du périmètre de cet exercice.

Retours booléens

Sauf mention contraire, les méthodes retournent un (`bool`) pour indiquer le succès ou l'échec. C'est un choix pragmatique pour un modèle simplifié. Une approche plus robuste utiliserait des exceptions pour forcer les appelants à gérer les cas d'erreur — un (`bool`) peut silencieusement être ignoré.

Les albums stockent des objets (`Song`), pas des clés étrangères

Toutes les références entre classes utilisent des objets plutôt que des identifiants bruts. Par exemple, (`Album.songs`) est typé (`set[Song]`), et non (`set[int]`). Cela maintient le modèle dans le **domaine orienté objet** et évite de faire fuiter les préoccupations de base de données relationnelle dans le diagramme de classes. La conversion en clés étrangères est une responsabilité de la couche de persistance, gérée de façon transparente par un ORM ou un repository.

2. Distribution des responsabilités

Classe	Responsabilité
StreamingPlatform	Façade : validation des entrées, orchestration, délégation
User	Gère ses propres playlists et les artistes suivis
Playlist	Gère sa propre collection de chansons
Album	Gère sa propre collection de chansons, avec méthodes publiques batch et méthodes privées unitaires
Artist	Représente un créateur musical ; possède des albums
Song	Unité musicale atomique ; appartient à un album et un artiste

Un principe clé est que `StreamingPlatform` **ne manipule jamais directement les données** — elle valide, puis délègue. Par exemple, `createPlaylist` vérifie le format du nom (prévention XSS) avant d'appeler `Playlist::create`. Cela maintient la logique métier à l'intérieur des classes de domaine.

3. Relations et multiplicités

Hiérarchie : Artist → Album → Song

```
Artist "1" *-- "0..*" Album : designs
Album "1" *-- "0..*" Song : contains
Song "1" --> "1" Artist : composed and interpreted by
```

- `Artist *-- Album` est une **composition** : un album ne peut pas exister sans son artiste.
- `Album *-- Song` est une **composition** : une chanson appartient à exactement un album dans ce modèle simplifié.
- `Song --> Artist` est une **association orientée** : une chanson référence son artiste de façon permanente. Cette relation crée un raccourci en parallèle du chemin `Album → Artist` — c'est intentionnel, pour accéder directement à l'artiste depuis une chanson sans traverser l'album.

Débat noté : nous avons initialement discuté de la hiérarchie `Artist → Song → Album` (reflétant le processus créatif) versus `Artist → Album → Song` (reflétant la structure des contenus publiés sur une plateforme de streaming). Nous avons retenu la seconde car elle représente mieux le **modèle de données publié**, et non le flux de création.

Agrégation vs composition pour `Album *-- Song` : dans un système réel, les chansons peuvent exister indépendamment (rééditions, compilations), ce qui en ferait une agrégation. Nous l'avons modélisée comme une composition par simplification, en notant explicitement ce compromis.

User et Playlist

```
User "1"      *-- "0..*" Playlist : has exclusive
User "0..*" o-- "0..*" Artist    : follows
```

- `User *-- Playlist` est une **composition** : dans ce modèle, une playlist est privée et appartient à exactement un utilisateur. Si des playlists partagées étaient introduites, cela deviendrait une agrégation avec une relation many-to-many.
- `User o-- Artist` est une **agrégation** : un artiste existe indépendamment de ses abonnés.

Sur la conception bidirectionnelle de `User ↔ Playlist`: `User` porte un attribut `playlists: set[Playlist]`, et `Playlist` porte un attribut `owner: User`. Cette bidirectionnalité est un choix délibéré pour la navigabilité — accès direct depuis chaque côté — et non une redondance de données. En base relationnelle, seul `Playlist.owner_id` existerait comme clé étrangère ; l'attribut `User.playlists` reflète la navigabilité de l'association, pas une colonne supplémentaire.

Playlist et Song

```
Playlist "1" o-- "0..*" Song : references
```

- **Agrégation** : une chanson existe indépendamment de toute playlist. Une playlist peut également être vide (ex : tout juste créée). Les deux côtés justifient le choix de l'agrégation plutôt que la composition.

StreamingPlatform

```
StreamingPlatform "1" --> "0..*" Album, Song, User, Playlist : uses
```

Ce sont des **associations orientées** : `StreamingPlatform` référence et manipule tous les objets de domaine, mais ne possède pas leur cycle de vie. Une composition impliquerait qu'aucun objet de domaine ne peut exister sans la plateforme, ce qui est sémantiquement incorrect. Aucune relation explicite vers `Artist` n'a été ajoutée, car aucune méthode de `StreamingPlatform` ne prend ou ne retourne directement un objet `Artist` — le filtrage lié aux artistes utilise des paramètres `str` bruts.

4. Alternatives envisagées

Classe de liaison `UserPlaylist`

Nous avons envisagé d'introduire une classe d'association `UserPlaylist` entre `User` et `Playlist`. Nous l'avons écartée car **la relation ne porte aucune donnée propre** (pas de `added_at`, pas de `role`, pas de `is_pinned`). Une classe de liaison ne se justifie que lorsque la relation elle-même possède des attributs. Sans cela, elle ne ferait que matérialiser une table de jointure SQL — un détail d'implémentation, pas un concept métier.

Elle deviendrait pertinente si des playlists collaboratives étaient introduites, car la relation devrait alors porter un attribut `role` (propriétaire, collaborateur, lecteur).

`song_id` vs objets `Song` comme paramètres de méthode

Nous avons envisagé de passer des valeurs brutes `song_id: int` à des méthodes comme `Playlist::addSong` et `Album::addSong`. Nous avons choisi de passer des objets `Song` complets pour maintenir le modèle au **niveau du domaine** et permettre la validation directement dans la méthode réceptrice. Passer des ids ferait remonter des préoccupations ORM dans la logique métier.

`Artist → Song → Album` vs `Artist → Album → Song`

Discuté ci-dessus (voir section 3). La chaîne `Artist → Album → Song` a été retenue car elle reflète l'**état publié** du contenu sur une plateforme de streaming, où une chanson est toujours diffusée dans le cadre d'un album (y compris les singles, qui sont techniquement des albums d'une seule piste).

`Album *-- Song` (**composition**) vs `Album o-- Song` (**agrégation**)

La composition a été choisie par simplification. En réalité, une chanson peut survivre à son album (rééditions, compilations, suppression d'album par l'artiste). Le commentaire dans le diagramme de classes reconnaît explicitement ce compromis.

Séparation `addSong` / `addSongs` sur `Album`

Nous avons choisi d'exposer à la fois une méthode publique batch `+addSongs(list[Song]): int` et une méthode privée unitaire `-addSong(Song): bool`. La méthode privée concentre toute la logique de validation (vérification des doublons, contrôle de format) ; la méthode publique batch itère simplement et lui délègue. Cela évite de dupliquer la logique de validation et rend l'API publique flexible.

5. Compromis

Décision	Avantage	Limite
Façade <code>StreamingPlatform</code> unique	Simple, lisible, centré sur le domaine	Ne représente pas une vraie architecture en couches
<code>create</code> / <code>get</code> sur les classes de domaine	Auto-suffisant, facile à lire	Mélange responsabilités domaine et persistance
Retours <code>bool</code>	Simple à modéliser et à lire	Les échecs peuvent être ignorés silencieusement ; les exceptions seraient plus sûres
Composition pour <code>Album</code> <code>*-- Song</code>	Propriété claire, cohérente avec le modèle simplifié	Ne reflète pas la flexibilité réelle (rééditions, compilations)
Composition pour <code>User</code> <code>*-- Playlist</code>	Simple, reflète la propriété privée	Devrait devenir une agrégation si des playlists partagées étaient introduites
Références par objets plutôt que par ids	Modèle OO propre	Nécessite une couche de traduction ORM en pratique
Pas de subdivision <code>Admin</code> / <code>Client</code>	Garde le modèle centré sur le périmètre de l'exercice	<code>roles: list[str]</code> sur <code>User</code> n'est qu'un espace réservé

6. Ce que nous améliorerions

- **Séparer la persistance du domaine** : extraire les méthodes `create`/`get` dans des classes `Repository` dédiées (`PlaylistRepository`, `SongRepository`, etc.) pour respecter le Principe de Responsabilité Unique.
- **Remplacer les retours `bool` par des exceptions** : forcer la gestion des erreurs au niveau de l'appelant plutôt que de compter sur lui pour vérifier les valeurs de retour.
- **Modéliser `Admin` séparément de `User`** : introduire un modèle de contrôle d'accès basé sur les rôles, ou a minima une sous-classe `Admin` appropriée, plutôt qu'un `roles: list[str]` brut.
- **Introduire un `Protocol` / interface** : les méthodes statiques communes (`create`, `get`, `delete`) pourraient être regroupées sous un protocole partagé `Persistable`, réduisant la redondance entre les classes.
- **Passer `Album o-- Song` en agrégation** : plus honnête vis-à-vis des contraintes du monde réel, au prix d'une sémantique de propriété légèrement plus souple.